

إجابات تمرين التفكير التطبيقي (سيناريو واقعي)

1. لماذا؟ fetch() هل ستستخدم.

نعم، ولكن بحذر وبتقنية مناسبة:

php

```
// ✗ خطأ: جلب كل البيانات ثم التصفية
$result = $conn->query("SELECT * FROM users");
$all_users = $result->fetch_all(MYSQLI_ASSOC); // 500,000 سطر في الذاكرة

// ✓ صحيح: جلب البيانات على دفعات
$stmt = $conn->prepare("SELECT id, name, email FROM users WHERE name LIKE ? OR
DER BY id LIMIT ? OFFSET ?");
$stmt->bind_param("sii", $search_pattern, $limit, $offset);
$stmt->execute();
$result = $stmt->get_result();

$users = [];
while ($row = $result->fetch_assoc()) {
    $users[] = $row; // 20 سطر فقط في الذاكرة
}
```

تتيح جلب البيانات على دفعات صغيرة بدلاً من تحميل `fetch()` أو `fetch_assoc()` السبب: كل النتائج في الذاكرة مرة واحدة.

2. LIMIT وOFFSET كيف ستستخدم.

استخدام فعال مع 500,000 مستخدم:

php

```
// حساب البجعة
$page = max(1, (int)$_GET['page'] ?? 1);
$limit = 20;
$offset = ($page - 1) * $limit;

// LIMIT وOFFSET استعمال أساسي مع
$sql = "SELECT id, name, email FROM users WHERE name LIKE ? LIMIT ? OFFSET ?";
```

```
// ✗ مشكلة: OFFSET البيانات الكبيرة بطيء جداً مع
// Cursor-Based Pagination الحل: استخدام تقنية
$last_id = (int)$_GET['last_id'] ?? 0;
$sql = "SELECT id, name, email FROM users
        WHERE name LIKE ? AND id > ?
        ORDER BY id
        LIMIT ?";
```

أي نوع سأختار؟ ولماذا؟ 3.

حلول متقدمة للبيانات الكبيرة:

php

```
// ✔ 1 الحل: Cursor-Based Pagination (الأفضل)
$last_id = (int)$_GET['last_id'] ?? 0;
$search_pattern = '%' . $_GET['search'] . '%';

$stmt = $conn->prepare("
    SELECT id, name, email
    FROM users
    WHERE (name LIKE ? OR email LIKE ?)
    AND id > ?
    ORDER BY id
    LIMIT 20
");
$stmt->bind_param("ssi", $search_pattern, $search_pattern, $last_id);
$stmt->execute();

// ✔ 2: فهرسة ذكية
// أنشئ فهرس مركب للبحث
// CREATE INDEX idx_users_search ON users(name, id);

// ✔ 3 الحل: Materialized Views للبحث المتكرر
// CREATE VIEW users_search_view AS
// SELECT id, name, email,
//        CONCAT(name, ' ', email) AS searchable
// FROM users;
```

4. SQL أم PHP: أين يجب تنفيذ التحقق من الإدخال

في كلا المكانين، ولكن لكل دوره

php

```
// ===== PHP: التحقق من المصيغة والقواعد =====
$search = trim($_GET['search'] ?? '');

// 1. التحقق من الطول
if (strlen($search) < 2) {
    die("اكتب حرفين على الأقل للبحث");
}

// 2. التحقق من الأحرف المسموح بها
if (!preg_match('/^[a-zA-Z0-9\u0600-\u06FF\s\-\_\.\@]+\$/u', $search)) {
    die("أحرف غير مسموح بها في البحث");
}

// 3. التحقق من الصفحة
$page = (int)($_GET['page'] ?? 1);
if ($page < 1) {
    die("رقم الصفحة غير صحيح");
}

// 4. التحقق من معدل الطلبات (Rate Limiting)
session_start();
$rate_limit_key = 'search_' . md5($_SERVER['REMOTE_ADDR']);
if (!isset($_SESSION[$rate_limit_key])) {
    $_SESSION[$rate_limit_key] = 0;
    $_SESSION['rate_limit_time'] = time();
}

if ($_SESSION[$rate_limit_key] > 10 && (time() - $_SESSION['rate_limit_time'])
< 60) {
    die("لقد تجاوزت الحد المسموح من عمليات البحث");
}

// ===== SQL: الأمان ضد الحقن =====
// SQL ، ولكن الأمان النهائي في PHP التحقق يحدث في
```

```
$search_pattern = '%' . $search . '%';  
$stmt = $conn->prepare("SELECT ... WHERE name LIKE ?");  
$stmt->bind_param("s", $search_pattern); // الحماية النهائية
```

5. SQL Injection? كيف تحمي الاستعلام من

طبقات حماية متعددة:

php

```
// ===== الحماية الكاملة =====  
  
// 1. Prepared Statements (الطبقة الأساسية)  
$search = $_GET['search'] ?? '';  
$page = (int)($_GET['page'] ?? 1);  
  
$stmt = $conn->prepare("  
    SELECT id, name, email  
    FROM users  
    WHERE name LIKE CONCAT('%', ?, '%')  
    ORDER BY id  
    LIMIT 20 OFFSET ?  
");  
  
// 2. Type Casting للرقم  
$offset = max(0, ($page - 1) * 20);  
$stmt->bind_param("si", $search, $offset);  
  
// 3. Stored Procedures طبقة إضافية  
$conn->query("  
    CREATE PROCEDURE search_users(  
        IN search_text VARCHAR(100),  
        IN page_num INT  
    )  
    BEGIN  
        DECLARE offset_val INT DEFAULT 0;  
        SET offset_val = (page_num - 1) * 20;  
  
        SELECT id, name, email  
        FROM users
```

```

        WHERE name LIKE CONCAT('%', search_text, '%')
        ORDER BY id
        LIMIT 20 OFFSET offset_val;
    END
");

```

// الاستخدام

```

$stmt = $conn->prepare("CALL search_users(?, ?)");
$stmt->bind_param("si", $search, $page);

```

// 4. Input Sanitization (التنظيف)

```

function sanitize_search($input) {
    // إزالة الأحرف الخطرة
    $input = strip_tags($input);
    $input = htmlspecialchars($input, ENT_QUOTES, 'UTF-8');

    // تقييد الطول
    $input = substr($input, 0, 100);

    // إزالة المسافات الزائدة
    $input = preg_replace('/\s+/', ' ', $input);

    return trim($input);
}

```

// 5. Full-Text Search (أكثر أماناً)

```

$conn->query("ALTER TABLE users ADD FULLTEXT(name, email)");

```

```

$stmt = $conn->prepare("
    SELECT id, name, email,
           MATCH(name, email) AGAINST(? IN BOOLEAN MODE) AS relevance
    FROM users
    WHERE MATCH(name, email) AGAINST(? IN BOOLEAN MODE)
    ORDER BY relevance DESC
    LIMIT 20 OFFSET ?
");

```

// 6. كود كامل مع جميع طبقات الحماية

```

class UserSearch {

```

```

private $conn;

public function __construct($conn) {
    $this->conn = $conn;
}

public function searchUsers($search_term, $page = 1) {
    // التحقق في PHP
    if (strlen($search_term) < 2) {
        return ['error' => 'اكتب حرفين على الأقل'];
    }

    // التنظيف
    $search_term = $this->sanitizeInput($search_term);
    $page = max(1, (int)$page);

    // Prepared Statement
    $stmt = $this->conn->prepare("
        SELECT id, name, email
        FROM users
        WHERE name LIKE CONCAT('%', ?, '%')
            OR email LIKE CONCAT('%', ?, '%')
        ORDER BY name
        LIMIT 20 OFFSET ?
    ");

    $offset = ($page - 1) * 20;
    $stmt->bind_param("ssi", $search_term, $search_term, $offset);

    $stmt->execute();
    $result = $stmt->get_result();

    return $result->fetch_all(MYSQLI_ASSOC);
}

private function sanitizeInput($input) {
    return preg_replace('/[^\p{L}\p{N}\s\-\_\.\@]/u', '', $input);
}
}

```

```
// الاستخدام
```

```
$search = new UserSearch($conn);
```

```
$results = $search->searchUsers($_GET['search'], $_GET['page']);
```